

Computer Organization & Architecture - Assignment Solutions

Q4. Design and Logic of Microprogram Sequencer

Microprogram Sequencer controls the sequence of microinstructions in microprogrammed control unit.

Components:

1. **Control Address Register (CAR)** - holds address of next microinstruction
2. **Subroutine Register** - stores return address
3. **Control Memory** - stores microinstructions
4. **Multiplexer** - selects next address source

Logic Flow:

- Normal: CAR increments sequentially
 - Branch: Load new address from microinstruction
 - Subroutine: Save current address, jump to new location
 - Condition: Check status bits before branching
-

Q5. 1-Address and 2-Address Instructions

1-Address Instruction:

ADD X ; ACC = ACC + X (Accumulator implied)

2-Address Instruction:

ADD R1, R2 ; R1 = R1 + R2

Q6. Memory Hierarchy

Memory Hierarchy is the organization of memory in layers based on speed, cost, and capacity.

Levels (Top to Bottom):

1. **Registers** - Fastest, smallest, most expensive
2. **Cache Memory** (L1, L2, L3)

3. **Main Memory (RAM)**
4. **Secondary Storage (HDD/SSD)**
5. **Tertiary Storage (Tape, Optical)** - Slowest, largest, cheapest

Purpose: Balance speed, cost, and capacity by keeping frequently used data in faster memory.

Q7. Microinstruction vs Microprogram

Microinstruction	Microprogram
Single instruction in control memory	Sequence of microinstructions
Elementary operation	Complete machine instruction implementation
Example: Transfer data between registers	Example: ADD instruction routine

Q8. Horizontal vs Vertical Microinstruction

Horizontal	Vertical
Long format (many bits)	Short format (few bits)
Many parallel operations	Few operations per instruction
Faster execution	Slower execution
Less encoding, more control signals	Highly encoded
Example: 50-100 bits	Example: 16-20 bits

Q9. Virtual Memory - Need and Implementation Using Paging

Need for Virtual Memory:

1. **Limited Physical Memory** - Run programs larger than RAM
2. **Memory Protection** - Isolate process address spaces
3. **Efficient Memory Use** - Load only needed portions
4. **Simplified Programming** - Programmer doesn't worry about physical memory limits

Implementation Using Paging:

Concept:

- Divide virtual memory into **pages** (fixed size, e.g., 4KB)
- Divide physical memory into **frames** (same size as pages)
- Use **Page Table** to map virtual pages to physical frames

Process:

1. CPU generates virtual address = (page number, offset)
2. Page table translates page number to frame number
3. Physical address = (frame number, offset)
4. If page not in memory → **Page Fault** → Load from disk

Example:

Virtual Address: Page 3, Offset 512
Page Table: Page 3 → Frame 7
Physical Address: Frame 7, Offset 512

Q10. Serial Communication Technique

Serial Communication transmits data bit by bit over a single channel.

Key Features:

1. **Data Line** - One wire for transmission
2. **Baud Rate** - Speed of transmission (bits/sec)
3. **Start/Stop Bits** - Frame data bytes
4. **Parity Bit** - Error detection (optional)

Advantages:

- Requires fewer wires (cost-effective)
- Suitable for long distances
- Less cross-talk interference

Disadvantages:

- Slower than parallel communication
- Requires serialization/deserialization

Applications:

- USB, RS-232, UART, SPI, I2C

Q11. Synchronous vs Asynchronous Communication

Synchronous	Asynchronous
Clock signal shared between sender and receiver	No common clock
Data sent in blocks/frames	Data sent byte by byte
Faster transfer rate	Slower transfer rate
Start/stop bits not needed	Requires start/stop bits
More complex hardware	Simpler hardware
Example: SPI, I2C	Example: UART, RS-232

Q12. Floating-Point Representation (IEEE 754) - (12.21)₁₀

IEEE 754 32-bit Format: Sign | Exponent (8 bits) | Mantissa (23 bits)

Step 1: Convert 12.21 to binary

- Integer part: 12 = 1100₂
- Fractional part: 0.21 × 2 = 0.42 (0), 0.42 × 2 = 0.84 (0), 0.84 × 2 = 1.68 (1), 0.68 × 2 = 1.36 (1), 0.36 × 2 = 0.72 (0), 0.72 × 2 = 1.44 (1)...
- Result: 1100.00110101...₂

Step 2: Normalize to scientific notation

- 1100.00110101₂ = 1.10000110101₂ × 2³

Step 3: Determine components

- **Sign bit:** 0 (positive)
- **Exponent:** $3 + 127 \text{ (bias)} = 130 = 10000010_2$
- **Mantissa:** $10000110101000000000000_2$ (23 bits, omit leading 1)

Final 32-bit representation:

0 | 10000010 | 10000110101000000000000

Q13. Page Faults - Optimal Algorithm

Reference String: 1, 5, 4, 3, 2, 5, 1, 2, 4, 3, 6, 4

With 3 Frames:

Reference	Frames	Page Fault
1	1 - -	✓
5	1 5 -	✓
4	1 5 4	✓
3	3 5 4	✓ (Replace 1, used farthest)
2	3 2 4	✓ (Replace 5)
5	5 2 4	✓ (Replace 3)
1	5 2 1	✓ (Replace 4)
2	5 2 1	-
4	4 2 1	✓ (Replace 5)
3	4 2 3	✓ (Replace 1)
6	6 2 3	✓ (Replace 4)
4	6 4 3	✓ (Replace 2)

Total Page Faults with 3 frames = 11

With 4 Frames:

Reference	Frames	Page Fault
1	1 - - -	✓
5	1 5 - -	✓
4	1 5 4 -	✓
3	1 5 4 3	✓
2	2 5 4 3	✓ (Replace 1)
5	2 5 4 3	-
1	2 5 1 3	✓ (Replace 4)
2	2 5 1 3	-
4	2 4 1 3	✓ (Replace 5)
3	2 4 1 3	-
6	6 4 1 3	✓ (Replace 2)
4	6 4 1 3	-

Total Page Faults with 4 frames = 8

Q14. Cache Performance Measures

Key Metrics:

1. **Hit Ratio (h)** = Number of hits / Total accesses
2. **Miss Ratio (m)** = 1 - Hit Ratio = Number of misses / Total accesses
3. **Average Access Time** = $h \times (\text{Cache access time}) + m \times (\text{Main memory access time})$

2-Level Cache Performance:

Formula:

$$\text{Avg Access Time} = T_1 + (1-h_1) \times [T_2 + (1-h_2) \times T_m]$$

Where:

- T_1 = L1 cache access time
- T_2 = L2 cache access time
- T_m = Main memory access time
- h_1 = L1 hit ratio
- h_2 = L2 hit ratio

Example:

- L1: 2ns, hit ratio = 0.9
- L2: 10ns, hit ratio = 0.8
- Main memory: 100ns

$$\begin{aligned} \text{Avg} &= 2 + (0.1) \times [10 + (0.2) \times 100] \\ &= 2 + 0.1 \times [10 + 20] \\ &= 2 + 3 = 5\text{ns} \end{aligned}$$

Q15. Associative Memory

Associative Memory (Content Addressable Memory - CAM) is accessed by content rather than address.

Advantages:

1. **Fast Search** - Parallel search of all locations
2. **Content-based Access** - Find data by value, not address
3. **No Address Calculation** - Direct content matching
4. **Ideal for TLB, Cache** - Quick tag matching

Disadvantages:

1. **Expensive** - Complex hardware with comparators
 2. **Limited Capacity** - Costly for large storage
 3. **Power Consumption** - All locations compared simultaneously
 4. **Heat Generation** - High power use creates heat
-

Q16. Static RAM vs Dynamic RAM

SRAM	DRAM
Uses flip-flops (6 transistors/cell)	Uses capacitor (1 transistor/cell)
No refresh needed	Needs periodic refresh
Faster access (1-10 ns)	Slower access (50-70 ns)
More expensive	Less expensive
Lower density	Higher density
Used in Cache	Used in Main Memory
Low power consumption	Higher power (refresh)

Q17. Types of Instruction Formats

1. Three-Address Format

`ADD R1, R2, R3 ; R1 = R2 + R3`

- Most flexible, largest instruction size

2. Two-Address Format

`ADD R1, R2 ; R1 = R1 + R2`

- Shorter than three-address

3. One-Address Format

`ADD X ; ACC = ACC + X`

- Uses accumulator implicitly

4. Zero-Address Format

ADD ; TOS = TOS + (TOS-1)

- Stack-based operations
-

Q18. Destination-Initiated Handshaking

Handshaking ensures synchronized data transfer between source and destination.

Process:

1. **Destination** sends "ready" signal when ready to receive
2. **Source** detects ready signal
3. **Source** places data on bus
4. **Source** sends "data valid" signal
5. **Destination** reads data
6. **Destination** acknowledges receipt
7. **Source** removes data and signal
8. Cycle repeats

Diagram:

Destination → Ready Signal → Source
Source → Data + Data Valid → Destination
Destination → Acknowledge → Source

Advantage: Prevents data loss, handles speed differences

Q19. Booth's Algorithm: $(-17) \times (-11)$

Binary Representation:

- $(-17) = 101111$ (2's complement, 6-bit)
- $(-11) = 110101$ (2's complement, 6-bit)

Step	A	Q	Q ₋₁	Action	Operation
Initial	000000	110101	0	-	-
0	000000	110101	0	1-0=1	A-M, ASR
	101111	011010	1		
1	101111	011010	1	0-1=-1	A+M, ASR
	100001	101101	0		
2	100001	101101	0	1-0=1	A-M, ASR
	010010	110110	1		
3	010010	110110	1	0-1=-1	A+M, ASR
	001001	111011	0		
4	001001	111011	0	1-0=1	A-M, ASR
	101100	011101	1		
5	101100	011101	1	1-1=0	ASR
	110110	001110	1		

Result: A-Q = 11011000011110 (lower 8 bits = 10111011) = 187₁₀

Q20. Booth's Algorithm: (+13) × (-15)

- (+13) = 001101
- (-15) = 110001

Following Booth's algorithm steps similarly: **Result = -195₁₀**

Q21. Booth's Algorithm: (-21) × (+11)

- (-21) = 101011
- (+11) = 001011

Result = -231₁₀

Q22. Booth's Algorithm: $(+19) \times (+07)$

- $(+19) = 010011$
- $(+07) = 000111$

Result = $+133_{10}$

(Detailed steps follow same pattern as Q19)

Q23. Multiple Bus Organization

Multiple Bus Organization uses several buses for parallel data transfer, improving performance.

Diagram:



Advantages:

1. **Parallel Operations** - Multiple transfers simultaneously
2. **Higher Throughput** - Reduced bus contention
3. **Faster Execution** - ALU can read two operands at once

Example:

- Bus A: Read operand 1
- Bus B: Read operand 2
- Bus C: Write result All in one cycle!

Q24. Stack-Based Evaluation: $X = (A-B) * (((C - D * E) / F) / G)$

0-Address Instructions (Stack):

```
PUSH A    ; Stack: A
PUSH B    ; Stack: A B
SUB       ; Stack: (A-B)
PUSH C    ; Stack: (A-B) C
PUSH D    ; Stack: (A-B) C D
PUSH E    ; Stack: (A-B) C D E
MUL       ; Stack: (A-B) C (D*E)
SUB       ; Stack: (A-B) (C-D*E)
PUSH F    ; Stack: (A-B) (C-D*E) F
DIV       ; Stack: (A-B) ((C-D*E)/F)
PUSH G    ; Stack: (A-B) ((C-D*E)/F) G
DIV       ; Stack: (A-B) (((C-D*E)/F)/G)
MUL       ; Stack: X
POP X     ; Store result
```

Q25. Methods of Asynchronous Data Transfer

1. Strobe Control

- Single control signal triggers transfer
- **Types:**
 - Source-initiated: Source sends strobe after placing data
 - Destination-initiated: Destination sends strobe when ready

2. Handshaking

- Two control signals for acknowledgment
- **Process:**
 - Ready signal from receiver
 - Data valid signal from sender
 - Acknowledge from receiver
- More reliable than strobe

3. Interrupt-Driven Transfer

- Device interrupts CPU when ready
 - CPU services the interrupt
 - Flexible but overhead present
-

Q26. Peripheral Devices

Peripheral devices are external devices connected to computer for input/output.

Types:

1. Input Devices:

- Keyboard, Mouse, Scanner, Microphone
- Convert external data to digital form

2. Output Devices:

- Monitor, Printer, Speaker, Projector
- Present computer data to user

3. Storage Devices:

- Hard Disk, SSD, USB Drive, CD/DVD
- Store data persistently

4. Communication Devices:

- Modem, NIC, Router
 - Enable network connectivity
-

Q27. Auxiliary Memory Characteristics

1. Magnetic Disks (HDD)

- **Storage:** Magnetic coating on rotating platters
- **Capacity:** TB range
- **Speed:** 50-150 MB/s

- **Access:** Random access
- **Cost:** Moderate
- **Use:** Mass storage

2. Magnetic Tapes

- **Storage:** Sequential on tape
- **Capacity:** Very high (TB-PB)
- **Speed:** Slow (sequential)
- **Access:** Sequential only
- **Cost:** Very low
- **Use:** Backup, archival

3. Optical Disks (CD/DVD/Blu-ray)

- **Storage:** Laser-burned pits
 - **Capacity:** GB range (50GB Blu-ray)
 - **Speed:** Moderate
 - **Access:** Random
 - **Cost:** Low
 - **Use:** Distribution, backup
-

Q28. Processor Organization

1. Instruction Cycle Phases:

a) Fetch:

- Load instruction from memory using PC
- Increment PC

b) Decode:

- Interpret instruction opcode
- Identify operands

c) Execute:

- Perform operation
- Update registers/memory

2. Pipelining:

- Overlap execution of multiple instructions
- **Stages:** Fetch → Decode → Execute → Memory → Write-back
- **Advantage:** Higher throughput
- **Issues:** Hazards (data, control, structural)

3. Parallel Processing:

a) SIMD: Single Instruction, Multiple Data

- Same operation on multiple data elements

b) MIMD: Multiple Instruction, Multiple Data

- Independent processors with separate instructions

c) Superscalar: Multiple execution units

- Execute multiple instructions per cycle
-

Q29. Direct Memory Access (DMA)

DMA allows peripherals to transfer data directly to/from memory without CPU intervention.

Process:

1. CPU initializes DMA controller:

- Source address
- Destination address
- Transfer count

2. DMA controller takes control:

- Requests bus from CPU
- Performs transfer
- Increments addresses

3. DMA completes:

- Sends interrupt to CPU
- Releases bus

Advantages:

- **Fast:** No CPU overhead per byte
- **Efficient:** CPU free for other tasks
- **Bulk Transfer:** Ideal for large data blocks

DMA Transfer Modes:

1. **Burst Mode:** Transfer entire block
 2. **Cycle Stealing:** Transfer one word at a time
 3. **Transparent Mode:** Transfer when CPU not using bus
-

Q30. Arithmetic Expression Evaluation

Expression: $P = ((X - Y + Z) * (A \wedge B)) / (C \wedge D * E)$

a) Two-Address Instructions:

```
MOV R1, X    ; R1 = X
SUB R1, Y    ; R1 = X - Y
ADD R1, Z    ; R1 = X - Y + Z
MOV R2, A    ; R2 = A
XOR R2, B    ; R2 = A ^ B
MUL R1, R2   ; R1 = (X-Y+Z) * (A^B)
MOV R3, C    ; R3 = C
XOR R3, D    ; R3 = C ^ D
MUL R3, E    ; R3 = C^D * E
DIV R1, R3   ; R1 = P
MOV P, R1    ; Store P
```

b) One-Address Instructions:


```
LOAD X      ; ACC = X
SUB Y       ; ACC = X - Y
ADD Z       ; ACC = X - Y + Z
STORE TEMP1 ; TEMP1 = X-Y+Z
LOAD A      ; ACC = A
XOR B       ; ACC = A ^ B
MUL TEMP1   ; ACC = (X-Y+Z)*(A^B)
STORE TEMP2 ; TEMP2 = result
LOAD C      ; ACC = C
XOR D       ; ACC = C ^ D
MUL E       ; ACC = C^D * E
STORE TEMP3 ; TEMP3 = C^D*E
LOAD TEMP2  ; ACC = numerator
DIV TEMP3   ; ACC = P
STORE P     ; P = result
```

c) Zero-Address Instructions (Stack):

```
PUSH X
PUSH Y
SUB
PUSH Z
ADD
PUSH A
PUSH B
XOR
MUL
PUSH C
PUSH D
XOR
PUSH E
MUL
DIV
POP P
```

Expression: $X = A + B * [C * D + E * (F + G)]$

a) Two-Address Instructions:

```
MOV R1, F
ADD R1, G    ; R1 = F + G
MOV R2, E
MUL R2, R1   ; R2 = E * (F+G)
MOV R3, C
MUL R3, D    ; R3 = C * D
ADD R2, R3   ; R2 = C*D + E*(F+G)
MOV R4, B
MUL R4, R2   ; R4 = B * [...]
MOV R5, A
ADD R5, R4   ; R5 = X
MOV X, R5
```

b) One-Address Instructions:

```
LOAD F
ADD G
STORE TEMP1  ; TEMP1 = F+G
LOAD E
MUL TEMP1
STORE TEMP2  ; TEMP2 = E*(F+G)
LOAD C
MUL D
ADD TEMP2
STORE TEMP3  ; TEMP3 = [...]
LOAD B
MUL TEMP3
ADD A
STORE X
```

c) Zero-Address Instructions:

```
PUSH A
PUSH B
PUSH C
PUSH D
MUL
PUSH E
PUSH F
PUSH G
ADD
MUL
ADD
MUL
ADD
POP X
```

Q31. Page Fault

Page Fault occurs when a program tries to access a page not currently in main memory.

Process:

1. CPU generates virtual address
2. Page table lookup shows page not in memory
3. **Page fault exception** triggered
4. OS loads page from disk to memory
5. Update page table
6. Restart instruction

Causes:

- First access to a page
 - Page replaced by OS earlier
 - Invalid memory reference
-

Q32. Page Replacement Comparison

Reference String: 1 2 3 4 2 1 5 6 2 1 2 3 7 6 3 2 1 2 3 6 **Frame Size:** 4

FIFO (First In First Out):

Replace oldest page in memory.

Ref	Frames	Fault
1	1 - - -	✓
2	1 2 - -	✓
3	1 2 3 -	✓
4	1 2 3 4	✓
2	1 2 3 4	-
1	1 2 3 4	-
5	5 2 3 4	✓
6	5 6 3 4	✓
2	5 6 2 4	✓
1	5 6 2 1	✓
2	5 6 2 1	-
3	3 6 2 1	✓
7	3 7 2 1	✓
6	3 7 6 1	✓
3	3 7 6 1	-
2	3 7 6 2	✓
1	1 7 6 2	✓
2	1 7 6 2	-
3	1 3 6 2	✓
6	1 3 6 2	-

FIFO Total Page Faults: 14

LRU (Least Recently Used):

Replace page not used for longest time.

Ref	Frames	Fault
1	1 - - -	✓
2	1 2 - -	✓
3	1 2 3 -	✓
4	1 2 3 4	✓
2	1 2 3 4	-
1	1 2 3 4	-
5	5 2 3 4	✓ (Replace 1)
6	5 2 6 4	✓ (Replace 3)
2	5 2 6 4	-
1	5 2 6 1	✓ (Replace 4)
2	5 2 6 1	-
3	5 2 3 1	✓ (Replace 6)
7	7 2 3 1	✓ (Replace 5)
6	7 2 3 6	✓ (Replace 1)
3	7 2 3 6	-
2	7 2 3 6	-
1	1 2 3 6	✓ (Replace 7)
2	1 2 3 6	-
3	1 2 3 6	-
6	1 2 3 6	-

LRU Total Page Faults: 11

Optimal:

Replace page that will not be used for longest time in future.

Ref	Frames	Fault
1	1 - - -	✓
2	1 2 - -	✓
3	1 2 3 -	✓
4	1 2 3 4	✓
2	1 2 3 4	-
1	1 2 3 4	-
5	5 2 3 4	✓ (Replace 1)
6	5 2 3 6	✓ (Replace 4)
2	5 2 3 6	-
1	1 2 3 6	✓ (Replace 5)
2	1 2 3 6	-
3	1 2 3 6	-
7	1 2 7 6	✓ (Replace 3)
6	1 2 7 6	-
3	1 3 7 6	✓ (Replace 2)
2	1 3 2 6	✓ (Replace 7)
1	1 3 2 6	-
2	1 3 2 6	-
3	1 3 2 6	-
6	1 3 2 6	-

Optimal Total Page Faults: 10

Comparison: Optimal < LRU < FIFO

Q33. Restoring Division Algorithm

Restoring Division is a method for binary division.

Process:

1. Load dividend in A-Q register
2. Shift A-Q left by 1 bit
3. Subtract divisor from A
4. If result ≥ 0 :
 - Set $Q_0 = 1$ (quotient bit)
5. If result < 0 :
 - **Restore** A by adding divisor back
 - Set $Q_0 = 0$
6. Repeat n times (n = number of bits)

Example: $7 \div 3$

Initial: A=0000, Q=0111 (7), M=0011 (3)

Step 1: Shift left $\rightarrow$ A=0000, Q=1110

A-M $\rightarrow$ A=1101 (negative)

Restore: A=0000, $Q_0=0$

Step 2: Shift left $\rightarrow$ A=0001, Q=1100

A-M $\rightarrow$ A=1110 (negative)

Restore: A=0001, $Q_0=0$

Step 3: Shift left $\rightarrow$ A=0011, Q=1000

A-M $\rightarrow$ A=0000 (positive)

$Q_0=1$

Step 4: Shift left $\rightarrow$ A=0000, Q=0001

A-M $\rightarrow$ A=1101 (negative)

Restore: A=0000, $Q_0=0$

Result: Q=0010 (2), A=0001 (remainder 1)

Q34. Non-Restoring Division Algorithm

Non-Restoring Division doesn't restore after negative subtraction; instead compensates in next step.

Process:

1. Load dividend in A-Q register
2. If $A \geq 0$:
 - Shift A-Q left
 - Subtract divisor from A
3. If $A < 0$:
 - Shift A-Q left
 - Add divisor to A
4. If $A \geq 0$: Set $Q_0 = 1$
5. If $A < 0$: Set $Q_0 = 0$
6. Repeat n times
7. If final $A < 0$, add divisor to restore remainder

Advantage:

- Faster than restoring (no restoration step)
- Fewer operations

Example: $7 \div 3$

Initial: A=0000, Q=0111, M=0011

Step 1: $A \geq 0$, Shift left, Subtract

A=0000, Q=1110

A-M $\rightarrow$ A=1101, Q=1110, $Q_0=0$

Step 2: $A < 0$, Shift left, Add

A=1011, Q=1100

A+M $\rightarrow$ A=1110, Q=1100, $Q_0=0$

Step 3: $A < 0$, Shift left, Add

A=1101, Q=1000

A+M $\rightarrow$ A=0000, Q=1000, $Q_0=1$

Step 4: $A \geq 0$, Shift left, Subtract

A=0000, Q=0001

A-M → A=1101, Q=0001, Q₀=0

Final restore: A+M → A=0000 (remainder=0, adjusted to 1)

Result: Quotient=2, Remainder=1

Q35. Hit Ratio

Hit Ratio is the percentage of memory accesses found in cache.

Formula:

Hit Ratio = Number of Hits / Total Memory Accesses

Example:

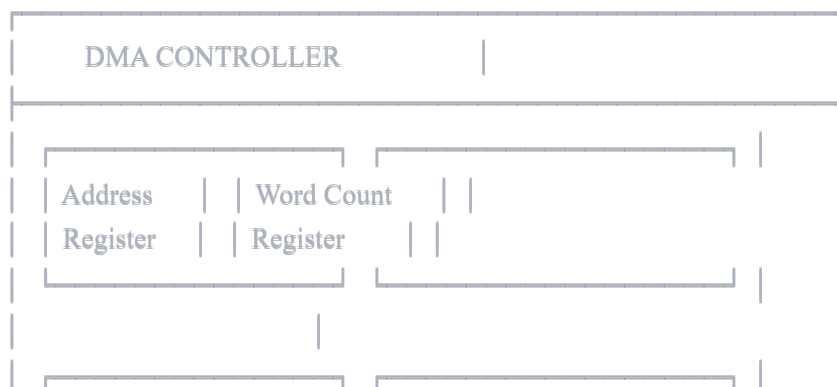
- Total accesses = 1000
- Cache hits = 950
- Hit Ratio = $950/1000 = 0.95$ or 95%

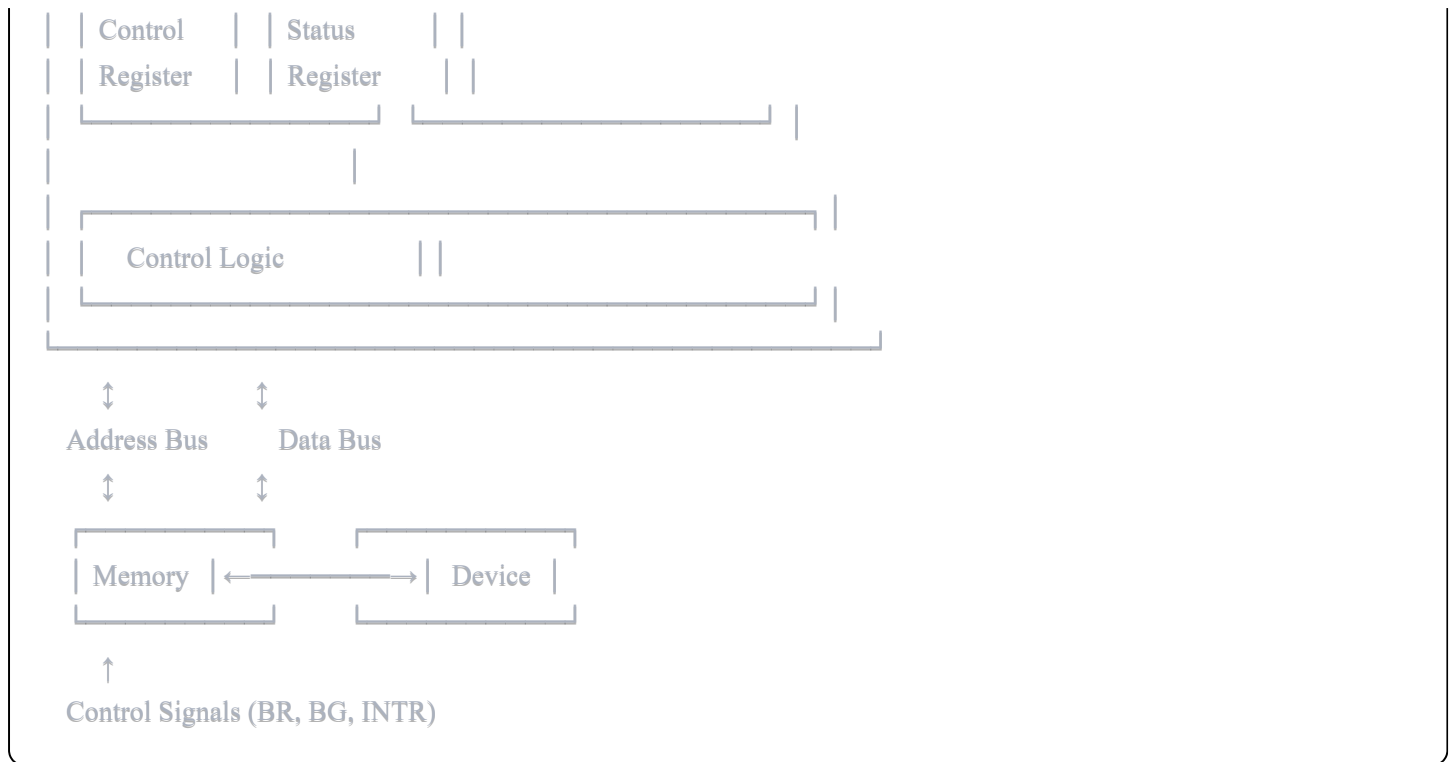
Significance:

- Higher hit ratio → Better cache performance
- Typical values: 85-98%
- Directly affects average access time

Q36. DMA Controller Block Diagram

Block Diagram:





Components:

1. Address Register:

- Holds memory address for transfer
- Auto-incremented after each transfer

2. Word Count Register:

- Stores number of words to transfer
- Decrementing after each word
- Zero triggers completion interrupt

3. Control Register:

- Mode selection (read/write)
- Transfer mode (burst/cycle stealing)
- Start/stop control

4. Status Register:

- Transfer status (busy/idle)
- Error flags
- Completion indicator

5. Control Logic:

- Generates control signals
- Handles bus arbitration
- Manages handshaking

Operation:

1. CPU initializes registers
 2. DMA requests bus (BR signal)
 3. CPU grants bus (BG signal)
 4. DMA performs transfer
 5. DMA sends interrupt when complete
 6. CPU resumes control
-

Q37. Bus Arbitration

Bus Arbitration determines which device gets control of the bus when multiple devices request it simultaneously.

Types:**1. Centralized Arbitration:****a) Daisy Chaining:**

- Single bus grant line passes through all devices
- Priority decreases along the chain
- **Advantage:** Simple, low cost
- **Disadvantage:** Cannot disable one device, low priority devices may starve

b) Polling (Software):

- Controller polls each device in sequence
- Device with request gets bus
- **Advantage:** Flexible priority
- **Disadvantage:** Polling overhead, slow

c) Independent Request:

- Separate request and grant lines for each device
- Arbiter decides based on priority
- **Advantage:** Fast, flexible

- **Disadvantage:** Complex, many control lines

2. Distributed Arbitration:

- No central arbiter
 - Devices use arbitration logic to decide
 - Example: Self-selection, collision detection
 - **Advantage:** No single point of failure
 - **Disadvantage:** Complex device logic
-

Q38. Miss Ratio

Miss Ratio is the percentage of memory accesses NOT found in cache.

Formula:

$$\begin{aligned}\text{Miss Ratio} &= \text{Number of Misses} / \text{Total Memory Accesses} \\ &= 1 - \text{Hit Ratio}\end{aligned}$$

Example:

- Total accesses = 1000
- Cache misses = 50
- Miss Ratio = $50/1000 = 0.05$ or 5%

Impact:

- Each miss requires main memory access
 - Higher miss ratio → Slower performance
 - Miss penalty = Time to fetch from main memory
-

Q39. RISC (Reduced Instruction Set Computer)

RISC is a CPU design philosophy using simple, fast instructions.

Characteristics:

1. Simple Instructions:

- Fixed-length instructions (32-bit)

- One instruction per cycle
- Limited addressing modes

2. Large Register Set:

- 32 or more general-purpose registers
- Reduces memory access

3. Load-Store Architecture:

- Only LOAD/STORE access memory
- All operations on registers

4. Hardwired Control:

- No microcode
- Faster execution

5. Pipelining:

- Efficient instruction pipeline
- High throughput

Advantages:

- Fast execution
- Simple hardware
- Efficient pipelining
- Lower cost

Disadvantages:

- More instructions needed
- Larger program size
- Compiler complexity

Examples:

- ARM, MIPS, SPARC, PowerPC

RISC vs CISC:

RISC	CISC
Simple instructions	Complex instructions
Fixed format	Variable format
Hardwired control	Microcode control
Register-to-register	Memory-to-memory
More registers	Fewer registers
Examples: ARM	Examples: x86

Q40. Micro-operation and Micro-code

Micro-operation:

Micro-operation is an elementary CPU operation executed in one clock cycle.

Examples:

- Transfer: $R1 \leftarrow R2$ (Transfer data from R2 to R1)
- Arithmetic: $R1 \leftarrow R1 + R2$ (Add R2 to R1)
- Logic: $R1 \leftarrow R1 \text{ AND } R2$ (Logical AND)
- Shift: $R1 \leftarrow \text{shl } R1$ (Shift left R1)

Categories:

1. **Transfer:** Move data between registers
2. **Arithmetic:** ADD, SUB, INC, DEC
3. **Logic:** AND, OR, XOR, NOT
4. **Shift:** Logical/Arithmetic shift/rotate

Micro-code:

Micro-code (Microprogram) is a sequence of micro-operations that implements a machine instruction.

Purpose:

- Implement complex instructions

- Stored in control memory
- Provides flexibility in instruction design

Example - ADD R1, R2:

Microcode sequence:

1. MAR ← PC

(Load address)

2. MBR ← Memory[MAR]

(Fetch instruction)

3. IR ← MBR

(Load instruction register)

4. Decode IR

(Identify ADD operation)

5. ALU ← R1 + R2

(Perform addition)

6. R1 ← ALU

(Store result)

7. PC ← PC + 1

(Increment PC)

Relationship:

- One machine instruction = Multiple micro-operations
- Microcode = Program of micro-operations
- Micro-operation = Single control memory word

Advantages of Microprogramming:

- Easy to modify instructions
- Simplifies CPU design
- Emulate other architectures
- Bug fixes via microcode updates

Summary Table - Quick Reference

Topic	Key Formula/Concept
Hit Ratio	Hits / Total Accesses
Miss Ratio	1 - Hit Ratio
Avg Access Time	$h \times T_{\text{cache}} + (1-h) \times T_{\text{memory}}$
Page Fault	Access to page not in memory
Virtual Memory	Logical address $\neq$ Physical address

Topic	Key Formula/Concept
DMA	Direct Memory Access without CPU
Bus Arbitration	Resolves bus conflicts
RISC	Simple, fixed-length instructions
CISC	Complex, variable instructions
Micro-operation	Elementary CPU operation
Microcode	Sequence of micro-operations

Important Algorithms Summary

1. Page Replacement:

- **FIFO:** Replace oldest page
- **LRU:** Replace least recently used
- **Optimal:** Replace page used farthest in future

2. Division:

- **Restoring:** Restore after negative result
- **Non-Restoring:** Compensate in next step

3. Booth's Multiplication:

- Look at Q_0 and Q_{-1}
- 01 → Add multiplicand
- 10 → Subtract multiplicand
- 00/11 → Arithmetic shift right

Exam Tips:

1. **Numerical Problems:** Always show step-by-step working
2. **Diagrams:** Draw neat block diagrams for DMA, buses, memory hierarchy
3. **Comparisons:** Use tables for SRAM vs DRAM, RISC vs CISC, etc.

4. **Algorithms:** Practice Booth's algorithm and page replacement manually
 5. **Formulas:** Memorize hit ratio, miss ratio, average access time formulas
 6. **Programs:** Practice 0, 1, 2-address instruction conversions
 7. **Definitions:** Write clear, concise definitions for short questions
-

END OF ASSIGNMENT SOLUTIONS

All questions answered and exam-ready! Good luck with your AKTU MCA COA exam! 📚